

Highlights

A Reproducible Edge-to-Cloud Infrastructure for Sustainable Orchestration of Embedded WebAssembly IoT Devices

Luca D'Agati, Giuseppe Tricomi, Fabio Orazio Mirto, Francesco Longo, Giovanni Merlino

- Kubernetes CRDs express intent for constrained non-node IoT devices.
- A TLS/CBOR gateway bridges cloud orchestration and embedded WASM execution.
- Reconciliation policies preserve evidence ownership across cloud, fog, and edge.
- A 100-trial campaign reports 100% transactional success (Wilson LB 96.3%) and 1185.8 ms mean end-to-end latency.
- Measured southbound control traffic is 1728 B/h per device; gateway mediation averages 18.2 MiB RAM.

A Reproducible Edge-to-Cloud Infrastructure for Sustainable Orchestration of Embedded WebAssembly IoT Devices

Luca D'Agati^{a,*}, Giuseppe Tricomi^a, Fabio Orazio Mirto^a, Francesco Longo^{a,b} and Giovanni Merlino^{a,b}

^aDepartment of Engineering, University of Messina, Messina, Italy

^bCINI: National Interuniversity Consortium for Informatics, Rome, Italy

ARTICLE INFO

Keywords:

Cloud–Fog–Edge continuum
digital research infrastructures
sustainable orchestration
green experimentation
Internet of Things
WebAssembly
embedded systems
Kubernetes

ABSTRACT

Cloud–Fog–Edge infrastructures increasingly need to deploy and observe application logic across cloud services, gateways, and embedded Internet of Things (IoT) devices, but conventional cloud-native orchestration assumes Linux-class nodes with container runtimes, resident agents, and sufficiently rich networking. This assumption leaves a gap for constrained endpoints that must remain lightweight while still requiring secure attachment, life-cycle accountability, and reproducible evidence for sustainable research infrastructures. This work addresses the problem of expressing Kubernetes deployment intent for devices that cannot operate as Kubernetes nodes and translating that intent into verifiable embedded execution. We propose a Kubernetes-native framework that models devices, applications, and gateways through Custom Resource Definitions (CRDs) and uses a gateway-mediated control path to bind device identity, supervise liveness, dispatch WebAssembly (WASM) workloads, and report execution evidence back to the control plane. The gateway acts as the semantic boundary between declarative cloud state and constrained device protocols, preserving device simplicity while maintaining status ownership in Kubernetes. The framework is evaluated on a K3s-based test bed with emulated embedded targets and a complete enrollment-to-deployment workflow. Across 100 independent trials, enrollment, heartbeat supervision, deployment, and transactional completion all succeeded (Wilson 95% lower bound 96.3%). Mean end-to-end latency is 1185.8 ms (95% CI 1152.8–1218.7 ms). Measured southbound control traffic is 1728 B/h per device at a 25 s heartbeat period; the gateway pod averages 18.2 MiB RAM and 3.6 millicores CPU with one TLS-connected device, while the device firmware occupies 394 KiB of flash and 264 KiB of RAM and carries no kubelet or container runtime. For sustainable digital research infrastructures, the framework therefore provides a reproducible, measurement-ready substrate for energy-aware placement, instrumentation, and green experimentation policies.

1. Introduction

Digital research infrastructures (RIs), cyber-physical systems, and Internet-of-Things (IoT) deployments increasingly require software-defined behavior at the edge, where constrained devices execute application logic under reliability, security, timing, life-cycle, and sustainability constraints. Remote deployments can include sensors, actuators, lightweight gateways, and mixed-criticality nodes that must remain inexpensive and portable while still being supervised continuously. In edge-to-cloud RIs, this supervision is not only an operational requirement but also a prerequisite for reproducible experimentation and future energy or carbon accounting across heterogeneous sites. Traditional embedded deployment practices, including manual flashing, board-specific provisioning, and tightly coupled update channels, are workable at small scale but do not provide fleet-wide orchestration or reliable visibility into which software version is running on each device.

In parallel, cloud-native ecosystems have converged around declarative orchestration, reconciliation loops, immutable artifacts, and service-level observability. Kubernetes has become the de facto orchestration substrate for distributed services because it separates desired state from

observed state and continuously drives the latter toward the former through controllers [39]. This model has transformed data-center operations, but it also exposes a gap when applied to embedded endpoints. Modern orchestrators assume Linux-capable nodes able to run containers, cluster agents, and rich networking stacks, whereas microcontrollers, sensors, and actuators generally cannot host these components. Nevertheless, cyber-physical deployments still need comparable guarantees for secure attachment, observability, and application life-cycle control.

WebAssembly (WASM) provides a practical execution abstraction for this gap. It supports portable, sandboxed workloads that can run across cloud, fog, and edge environments with a more stable module format than architecture-specific binaries [19]. For embedded systems, WebAssembly Micro Runtime (WAMR) and Zephyr-compatible integrations provide a concrete path toward microcontroller-oriented execution [9, 40]. Runtime feasibility alone, however, does not solve orchestration: a cloud-side intent must still be translated into a secure command for a constrained device, and execution evidence must be reflected back into the control plane.

This paper introduces a full-stack framework that addresses this problem through a gateway-centric architecture for the Cloud–Fog–Edge continuum. The framework builds an end-to-end path from Kubernetes CRDs on K3s, through

*Corresponding author.

Email address: ldagati@unime.it (L. D'Agati)

ORCID(s):

a Rust/Transport Layer Security (TLS) gateway that mediates identity validation and state propagation, to Zephyr firmware running WAMR on embedded-class targets. In this path, the cloud records what should happen, the gateway translates and validates that intent, and the device executes the WASM workload while reporting heartbeat-driven liveness and runtime outcomes. Embedded nodes therefore remain lightweight while still participating in a declarative orchestration model. For research infrastructures, the same path provides stable observation and actuation points that can be reused by sustainability dashboards, energy-aware schedulers, or green experiment-design workflows without moving cloud-native complexity into device firmware.

The work is motivated by a concrete engineering observation: secure enrollment is necessary but insufficient. A practical platform must also define ownership of status updates, avoid false disconnection semantics during transient conditions, and expose enough evidence for operators to trust the control plane. This requirement defines the central novelty of the work: a Kubernetes-native evidence and reconciliation model for non-node embedded WASM endpoints, combining declarative resource modeling, secure gateway mediation, state reconciliation, and deployment reproducibility into a single verifiable operational path.

1.1. Scope and Contributions

This work presents a systems-oriented contribution at the intersection of IoT orchestration, embedded computing, and secure cyber-physical infrastructure. It does not claim novelty in WebAssembly, Kubernetes, TLS, or Concise Binary Object Representation (CBOR) as isolated technologies. Instead, it integrates these elements into a coherent platform that closes an implementation gap frequently acknowledged in the literature but less often documented end to end: how declarative cloud-side intent can be made operational on constrained, non-node embedded devices without sacrificing secure attachment, control-plane visibility, and state fidelity.

The contributions are fourfold:

- a Kubernetes-native resource model with CRDs for devices, gateways, and deployable WASM applications (Section 4);
- a gateway-mediated TLS/CBOR enrollment and control workflow with explicit identity matching and southbound message translation (Section 5);
- a reconciliation strategy that links desired and reported state while avoiding false disconnect loops and ambiguous status ownership (Section 5);
- an end-to-end K3s validation study with emulated embedded targets and sustainability-oriented baselines for wire traffic, firmware footprint, and control-plane resource use (Section 7).

The contribution is positioned as an enabling layer for sustainable digital RIs: it does not yet optimize energy use, but it makes edge-device participation declarative, repeatable, and instrumentable for subsequent power, carbon, and placement studies.

The remainder of the paper is organized as follows. Section 2 reviews related work. Section 3 formulates the problem and design requirements; Section 4 presents the architecture and resource model. Section 5 describes the southbound protocol and reconciliation policy; Section 6 summarizes the implementation stack. Section 7 reports the experimental evaluation. Section 8 discusses implications, validity, and reproducibility. Section 9 concludes the paper.

2. Related Work

This section reviews prior work along the axes that motivate the design in Sections 3–6: edge orchestration, constrained execution, secure IoT communication, reconciliation semantics, and reproducible evaluation. The main gap across these areas is the lack of an end-to-end path from Kubernetes-style declarative intent to verifiable execution on constrained embedded devices that cannot act as ordinary cluster nodes.

2.1. Cloud-Native Edge and IoT Orchestration

Foundational work on fog and edge computing established the need to move computation, storage, and control closer to data sources in order to reduce latency, improve locality, and support cyber-physical applications [5, 11, 34, 38]. Kubernetes has become the dominant orchestration substrate for cloud-native services because it provides declarative resource management, controllers, reconciliation loops, and extensibility through Custom Resource Definitions (CRDs) and operators [22, 39]. Lightweight distributions such as K3s and MicroK8s reduce deployment overhead and make cluster-style operation feasible at smaller sites [10, 30].

Several systems extend Kubernetes semantics toward the edge. KubeEdge moves parts of Kubernetes control closer to edge nodes, OpenYurt targets cloud-edge application management, Akri discovers and exposes leaf devices to Kubernetes, and Krustlet explored WebAssembly workloads as Kubernetes-scheduled units [1, 20, 21, 28]. Recent Kubernetes-native IoT orchestration based on Stack4Things (S4T), CRDs, and Crossplane abstractions similarly demonstrates the value of declarative resource management for IoT deployments [13]. These approaches motivate the present work, but they generally assume Linux-class edge hosts, gateway nodes, container-compatible execution environments, device plugins, or platform services attached to capable nodes. The target considered here is narrower and more constrained: an embedded endpoint that cannot run a kubelet, container runtime, Kubernetes agent, or full cloud-native network stack.

IoT middleware and industrial edge platforms provide additional context. EdgeX Foundry, FIWARE, oneM2M, OMA Lightweight M2M, and OPC UA address aspects of device modeling, interoperability, telemetry, and industrial communication [15, 18, 25–27]. These frameworks are broader than a single orchestration mechanism and often emphasize interoperability or application enablement rather than Kubernetes-native desired-state reconciliation.

The contribution is therefore not another general IoT middleware layer, but a focused integration that links cloud-style intent, gateway-mediated secure control, and embedded WASM execution through explicit resource ownership.

2.2. Constrained Execution and Secure Communication

Embedded and IoT operating systems provide the substrate on which constrained devices communicate and execute application logic. Zephyr, FreeRTOS, RIOT, and Contiki-NG represent different points in the design space for real-time behavior, connectivity, portability, and resource footprint [2, 4, 14, 40]; TinyOS established the long-standing importance of event-driven and resource-aware design in sensor-network systems [23]. These operating systems make embedded applications feasible, but they do not by themselves provide a cloud-side mechanism for expressing intent, binding identity, reconciling state, and verifying deployment outcomes.

WebAssembly adds a portable execution layer that can complement embedded operating systems. The original WebAssembly design emphasized safe, compact, and efficient portable code [19, 44], while the WebAssembly System Interface (WASI) extended the ecosystem toward non-browser execution [8]. Runtime implementations such as WAMR, Wasmtime, WasmEdge, and wasm3 demonstrate that WASM can execute outside browsers and, in some cases, in resource-constrained settings [7, 9, 42, 43]. Container and cloud ecosystems have also begun to integrate WASM through projects such as runwasi and Spin [12, 17]. These efforts establish runtime feasibility, but not operational orchestration: the unresolved question is how a cloud control plane can safely decide that a device should run a given module, transmit that intent through a trusted gateway, and receive evidence that execution and liveness match the requested state.

Secure IoT communication relies on standardized transport, object security, and compact serialization mechanisms. TLS 1.3 and DTLS 1.3 provide channel security [31, 32]; CoAP and MQTT address lightweight application-layer communication [24, 37]; OSCORE and COSE provide object security and compact cryptographic containers [35, 36]; and CBOR supports low-overhead binary serialization [6]. These mechanisms are complementary to the present work. A secure channel can protect messages in transit, but it does not determine which component owns reported state, when a device should be considered connected, or how a cloud resource should converge after a deployment acknowledgment.

2.3. Reconciliation, Evaluation, and Positioning

The operator pattern and Kubernetes controller model encode desired state, observe actual state, and repeatedly reconcile the two [22, 39]. In conventional clusters, this model works because nodes and workloads participate directly in the Kubernetes ecosystem. Edge systems complicate the model because connectivity can be intermittent, observations may be delayed, and devices may expose

limited introspection. Prior platforms address parts of this challenge through edge agents, device twins, or local gateways [1, 15, 21, 27, 28]. For embedded targets, however, the gateway becomes more than a message forwarder: it is the semantic boundary between cloud resources and protocol-derived evidence.

Reproducible evaluation is equally important because physical hardware availability, network conditions, and manual setup can make embedded IoT experiments difficult to repeat. Large-scale experimental infrastructures such as SLICES highlight the role of programmable, shared scientific instruments for networking research [16]. Renode supports hardware-faithful emulation and multi-node embedded scenarios, while Cooja, ns-3, and OMNeT++ support repeatable simulation and network experimentation in wireless, sensor, and IoT research [3, 29, 33, 41]. The evaluation in this paper follows that direction by using an emulation-driven validation path and structured measurement records. This is appropriate because the main claim concerns end-to-end orchestration coherence rather than radio performance, energy consumption, or microarchitectural timing.

Taken together, existing cloud-edge orchestration systems extend Kubernetes toward capable edge nodes; IoT middleware improves interoperability and telemetry; embedded operating systems and WASM runtimes make constrained execution feasible; and secure communication standards provide protected message exchange. The proposed framework connects these strands at a narrower integration point: a Kubernetes-native control plane expressing declarative deployment intent for an embedded-class endpoint that remains outside the cluster and reports execution evidence through a gateway. Unlike middleware-centric approaches, the main abstraction is not a device model or telemetry bus, but the convergence between CRD intent, gateway evidence, and embedded WASM execution. The contribution is therefore best understood as a systems-integration study that combines explicit identity binding, gateway-mediated control, heartbeat-based liveness, CRD status convergence, and reproducible quantitative validation.

3. Problem Formulation and Design Requirements

The platform addresses the following problem:

How can a Kubernetes-based control plane reliably express the intent that a constrained device should execute a specific WebAssembly module, route that intent through a secure gateway, and verify the resulting execution and liveness state—with measurable control-plane and southbound overhead compatible with sustainable research-infrastructure operation—without requiring the device to behave like a Kubernetes node?

Cloud orchestrators normally assume Linux-capable workers with container runtimes, cluster agents, and rich

networking support. Embedded devices instead provide limited compute, memory, and network functionality, but they still need comparable guarantees for identity, observability, and life-cycle accountability. The framework therefore treats orchestration as an end-to-end translation problem between cloud intent, gateway-mediated protocol control, and edge-side WASM execution.

3.1. Design Tensions

Three tensions structure the problem. At the resource-model level, Kubernetes assumes a declarative control plane with explicit status ownership, whereas embedded deployments often rely on imperative, device-local procedures that are opaque to the cloud. At the transport level, constrained devices need lightweight and secure communication paths, but the cloud side requires rich enough semantics to express enrollment, liveness, and application control. Operationally, even when protocol messages are correct, reconciliation logic must interpret transient conditions conservatively to avoid reporting an incorrect device state.

3.2. Design Requirements

These tensions lead to a compact set of requirements. Desired and observed state for devices, applications, and gateways must remain represented in Kubernetes resources. Enrollment must occur over a protected channel with auditable binding between transport-level identity and platform-level resource identity. The application life cycle must use a format usable across cloud, fog, and constrained edge targets, motivating WebAssembly as a common execution abstraction. Runtime evidence must confirm enrollment, heartbeat-based liveness, deployment status, and gateway association without manual inspection of device internals. For digital research infrastructures, the same evidence model should expose where workload placement, liveness, and execution outcomes can later be correlated with site-level power or carbon signals. Finally, the complete stack must deploy and test in a commodity research environment.

Secure attachment without unified state would remain difficult to operate; unified state without secure attachment would be insecure by construction; and either property without runtime evidence would leave operators unable to trust the control plane. Communication, orchestration, and status management are therefore co-designed rather than layered opportunistically. Embedded devices are not trusted to manage cluster state directly, and Kubernetes is not expected to maintain device sessions directly: a managed gateway acts as the trusted fog-layer mediation point while the control plane remains authoritative for desired state.

4. Architecture

4.1. Layered View

The framework is organized around the Cloud–Fog–Edge continuum. The **cloud layer** hosts the API server, dashboard, controllers, CRDs, and supporting services on Kubernetes; it stores desired state and persists reported execution state. The **fog layer** hosts the Rust-based gateway

Table 1

Primary CRDs and control/evidence ownership.

Resource	Desired or configured state	Reported evidence
Device	Platform identity, expected key, gateway preference, runtime attributes	Gateway association, phase, connectivity, heartbeat freshness
Application	Target device, WASM artifact, desired execution state	Deployment phase, start outcome, execution result or error evidence
Gateway	Endpoint metadata, capabilities, namespace-scoped mediation path	Availability metadata, resolved devices, status propagation context

and emulation support services; it translates orchestration events into southbound TLS/CBOR messages, validates device identity, and propagates status updates back to the cloud. The **edge layer** contains Zephyr firmware, the WAMR execution environment, and deployed WASM modules; it executes the intended behavior and emits heartbeat and runtime evidence.

This decomposition localizes complexity where it is most affordable: Kubernetes retains declarative control, the gateway absorbs protocol and identity mediation, and the device remains lightweight. Figure 1 summarizes this path: declarative intent flows from the cloud toward the edge, while runtime evidence returns through the gateway into Kubernetes status fields.

4.2. Gateway-Centric Control

Gateway-centric mediation is the main architectural choice. Devices do not manipulate Kubernetes resources or embed cluster-specific logic. Instead, the gateway terminates southbound TLS sessions, validates enrollment messages, binds transport sessions to device identities, dispatches application life-cycle commands, and reports execution and liveness evidence to the cloud side.

This design keeps constrained devices small while concentrating protocol policy, identity checks, message normalization, and status propagation at a controllable boundary. It also gives the cloud side a stable integration surface if the device protocol evolves. Gateway correctness and availability are therefore explicit managed properties of the fog layer, rather than implicit assumptions distributed across constrained endpoints.

4.3. Resource Model

The framework relies on three primary CRDs, summarized in Table 1. The *Device* resource stores identity, gateway preference, runtime attributes, expected public key, and reported connectivity state. The *Application* resource expresses the desired WASM deployment for a target device and records execution outcomes in status fields. The *Gateway* resource describes endpoint metadata, capabilities, and the information needed to resolve the mediation path.

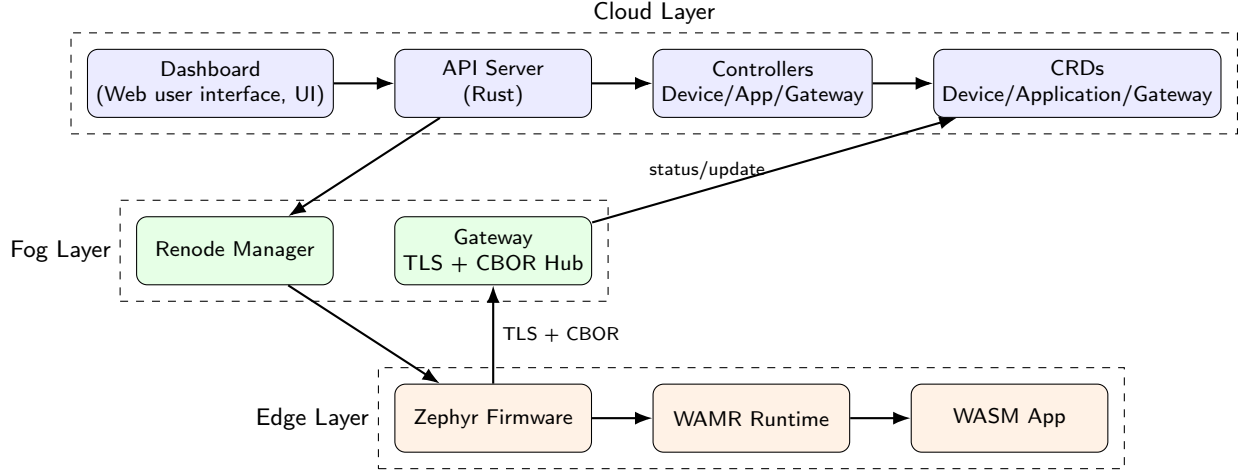


Figure 1: High-level architecture of the proposed framework. Declarative cloud intent is mediated by the gateway and translated into verifiable embedded WASM execution.

The resource model separates desired state from reported state. This preserves deployment intent when devices are offline, unreachable, or in transition, and it helps operators compare requested state with the latest evidence observed by the gateway and controllers.

4.4. Control and Observability Boundaries

The architecture distinguishes control ownership from evidence ownership. Operators and higher-level services change desired state through Kubernetes resources; the gateway turns protocol-level observations into platform-level status updates; and controllers enforce reconciliation and consistency. This separation reduces ambiguous responsibility and limits conflicting status writes. The validation campaign specifically exercises this boundary by checking that transient visibility gaps do not overwrite fresh gateway evidence or produce false disconnection state.

5. Protocol and Reconciliation

5.1. Southbound Life-Cycle

The southbound protocol uses framed CBOR messages over TLS. It is compact enough for embedded-class clients, but still supports identity binding, acknowledgments, liveness evidence, and application deployment. The life-cycle has three phases: enrollment, heartbeat-based supervision, and resource-driven WASM deployment.

Enrollment establishes the transition from a protected transport session to a platform-recognized device identity:

1. The device opens a protected session with the gateway and requests enrollment.
2. The gateway validates the request before proceeding.
3. The device provides identity material that can be bound to a known platform resource.
4. The gateway compares that material with the identity registered in the control plane.

5. The device confirms that enrollment information has been received and that it is ready for managed operation.
6. The gateway stores the device-session association and updates the corresponding resource status.

After enrollment, periodic heartbeats maintain liveness visibility. At the communication layer, they show that the protected channel and device remain responsive. At the control-plane layer, they provide evidence used to refresh reported connectivity and device phase. Missing or delayed observations are interpreted conservatively, so transient gaps do not immediately become disconnection events.

WASM deployment follows the same evidence-driven pattern, but starts from declarative intent rather than from device attachment:

1. An operator or higher-level service declares that a target device should run a specific WASM workload.
2. The control plane observes the desired state and identifies the mediation path through the appropriate gateway.
3. The gateway translates the deployment intent into compact protected messages for the device.
4. The device firmware uses its embedded WASM runtime to load and start the workload.
5. The device returns execution evidence or an error indication to the gateway.
6. The gateway propagates the reported outcome back into the control-plane status visible to operators.

Figure 2 summarizes the deployment loop: the controller resolves intent, the gateway mediates the southbound command, the device reports an outcome, and the control plane reconciles status. This preserves the declarative pattern and remains compatible with retry, staged rollout, and multi-gateway scheduling policies.

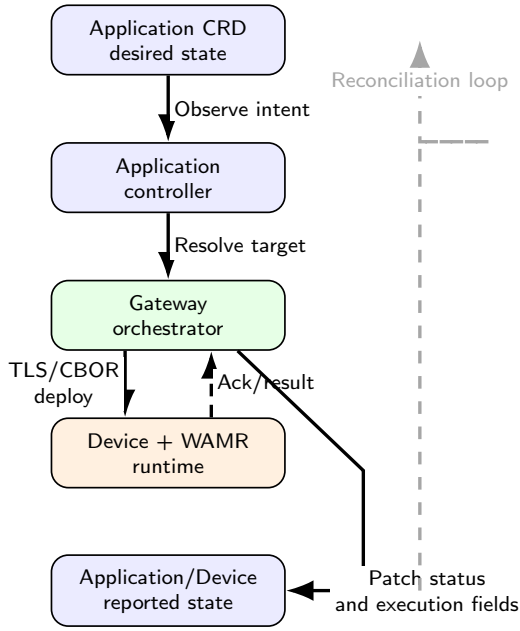


Figure 2: Application deployment workflow and status-reconciliation loop.

5.2. Protocol Sequences

Figures 3 and 4 summarize the southbound protocol interactions evaluated in Section 7. The diagrams emphasize role ownership: the device initiates attachment and emits runtime evidence, the gateway validates and translates protocol events, and the Kubernetes control plane stores desired and reported state.

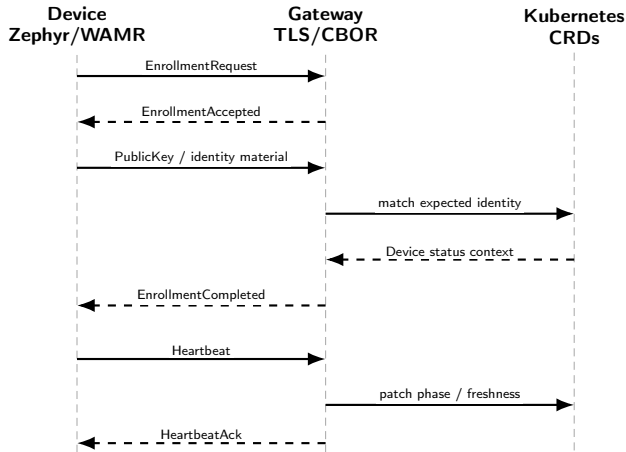


Figure 3: Enrollment and heartbeat sequence. Device-originated evidence is validated by the gateway and reflected into Kubernetes status.

During enrollment, the decisive event is not the opening of a protected transport session alone, but the successful association between the device-provided identity material and the Device resource expected by the control plane. Heartbeats then refresh the same association by producing liveness evidence that the gateway can interpret before patching Kubernetes status. Deployment follows the complementary

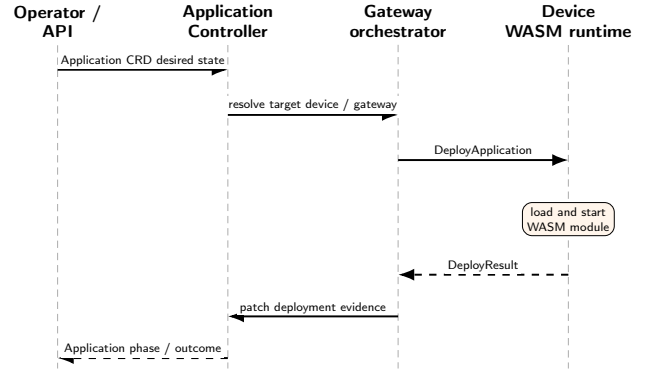


Figure 4: WASM deployment sequence. Declarative application intent is translated by the gateway into device execution and reconciled as application status.

direction: the Application resource records desired state in Kubernetes, the controller resolves the target and mediation path, and the gateway converts that intent into a compact southbound command. The returned DeployResult is treated as execution evidence, not merely as a transport acknowledgment: only after this evidence is propagated does the application status converge to the reported outcome.

5.3. Reconciliation Policy

Protocol correctness becomes platform correctness only when message outcomes are interpreted consistently by the control plane. The framework therefore gives the gateway authority over protocol-derived observations and constrains controllers to reconcile those observations without overreacting to transient visibility gaps.

Integration testing exercised short-lived differences between gateway observations and control-plane status, confirming that liveness should not be inferred from a single incomplete observation. The resulting policy preserves the latest valid gateway association, avoids immediate disconnection when heartbeat evidence and transport visibility briefly diverge, and waits for a subsequent reconciliation cycle before treating the condition as a real state transition. Reconciliation policy is therefore part of the architecture rather than an incidental implementation detail.

6. Implementation

The implementation combines Rust control-plane services with Zephyr-based device firmware and WAMR execution. The API server, gateway, and controllers are implemented in Rust, using its memory-safety guarantees and asynchronous networking support. Resource interaction follows Kubernetes controller and status-patching practices [22, 39], allowing embedded endpoints to be managed as first-class resources without treating them as cluster nodes. On the device side, Zephyr provides the embedded networking and TLS substrate [40], while WAMR provides lightweight WASM execution on constrained targets [9].

Table 2

Experimental testbed and software configuration.

Item	Configuration
Host OS	Ubuntu 24.04 LTS (kernel 6.8.0), x86_64 KVM guest
Host resources	32 vCPUs, 62 GiB RAM, 293 GiB SSD
Orchestrator	K3s v1.35.4+k3s1, single control-plane node
Southbound emulation	Renode antmicro/renode:nightly, board Stm32F746gDisco
Firmware stack	Zephyr RTOS 3.5.0, WAMR 2.4.1, minimal WASM test module (33 B)
Device networking	TAP tap0 at 192.168.1.1/24, DHCP, DNAT to gateway TLS on port 30443

The reference deployment runs on K3s [30]. It uses namespace-scoped resources and locally built images, preserving real Kubernetes API and reconciliation semantics while remaining accessible on a single-node Linux environment for development, experimentation, and repeatable evaluation. The deployment includes the API services, dashboard, gateway/controller components, CRDs, and supporting services needed to exercise enrollment, deployment, and status propagation.

Responsibilities are distributed across specialized controllers rather than concentrated in a single service. Device-oriented reconciliation manages attachment state and gateway resolution. Application-oriented reconciliation interprets deployment intent and maps it to gateway actions. Gateway-oriented reconciliation maintains metadata needed for path resolution and status interpretation. This decomposition follows the CRD model and reduces the risk that unrelated concerns become coupled inside one control loop.

7. Experimental Evaluation

7.1. Evaluation Design and Testbed

The evaluation combines (i) a functional validation campaign over the full enrollment-to-deployment path and (ii) a sustainability-oriented measurement pass on the same testbed. The functional study asks whether cloud intent, gateway mediation, and embedded WASM execution produce mutually consistent evidence under repeatable conditions. The sustainability pass quantifies southbound wire sizes, edge firmware footprint, and Kubernetes pod resource use. Neither pass claims measured joules or carbon reductions; they establish baselines relevant to green-experimentation follow-up.

Experiments run on a single-node K3s cluster with API server, gateway, application controller, and CRDs active. The southbound endpoint is a Renode-emulated STM32F746 running Zephyr firmware with WAMR; it connects over TAP-based networking with DNAT to the gateway TLS port. Table 2 summarizes the host and software stack.

After one warm-up run, the campaign executes $n=100$ independent trials. Each trial performs enrollment verification, heartbeat supervision, and deployment of the minimal WASM module to the same emulated device. A trial is

Table 3Latency profile ($n=100$ trials). All values in milliseconds.

Stage	Mean	Median	p95	p99	IQR	CV	95% CI of mean
Enrollment	583.9	557.4	773.4	791.1	58.8	0.142	[567.5, 600.3]
Heartbeat obs.	4.43	4.31	5.79	7.05	0.73	0.153	[4.30, 4.57]
Deployment	601.9	542.8	1031.3	1295.7	88.0	0.288	[567.5, 636.3]
End-to-end	1185.8	1132.5	1561.1	1789.6	191.5	0.140	[1152.8, 1218.7]

transactionally successful only if all three stages succeed and deployment evidence is consistent across the gateway runtime view and the Application CRD (phase=Running, device connected, fresh heartbeat). Continuous latencies use monotonic client-side timers; we report mean, standard deviation, median, p95, p99, IQR, CV, and 95% CI of the mean ($\bar{x} \pm t_{0.975, n-1} s / \sqrt{n}$, with $t_{0.975, 99}=1.984$). For binary outcomes we use Wilson score intervals; with 100/100 successes the 95% lower bound is 96.3%.

7.2. Functional Validation Results

All 100 transactional trials completed successfully: enrollment, heartbeat supervision, deployment, and cross-layer evidence checks never failed. Because every stage shares the same flawless outcome, a per-stage success table would repeat identical Wilson bounds and is omitted; the meaningful variation lies in latency and its distribution.

Table 3 reports the latency profile. Mean end-to-end latency (enrollment plus deployment; heartbeat excluded as a sub-millisecond probe) is 1185.8 ms with 95% CI [1152.8, 1218.7] ms. The heartbeat row reflects *observability* latency—how quickly the measurement harness reads a fresh heartbeat from the gateway—not the 25 s firmware emission interval.

Figure 5 compares stage latency distributions on a logarithmic scale. Heartbeat observability is two orders of magnitude faster than enrollment or deployment. Figure 6 shows that deployment contributes most of the tail mass: its p95 (1031.3 ms) and p99 (1295.7 ms) exceed enrollment's, so WASM orchestration—not TLS enrollment—dominates end-to-end variability (deployment CV = 0.288 vs. enrollment CV = 0.142).

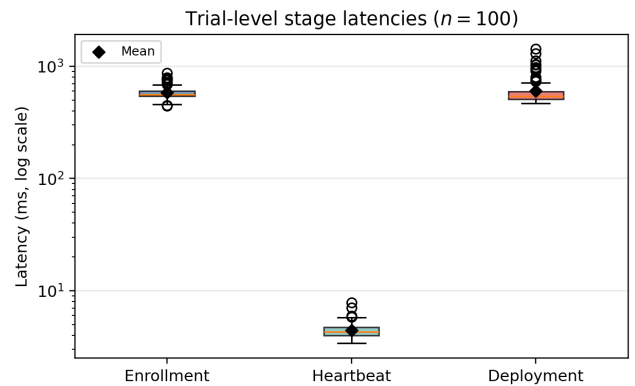


Figure 5: Distribution of trial-level stage latencies ($n=100$, log scale). Diamonds indicate sample means.

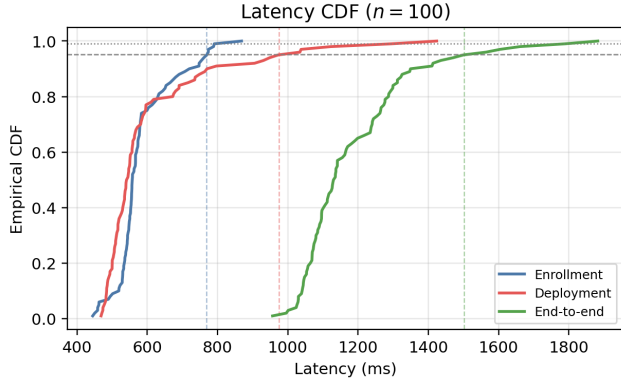


Figure 6: Empirical CDF of enrollment, deployment, and end-to-end latencies ($n=100$).

7.3. Sustainability-Oriented Metrics

7.3.1. Southbound wire efficiency

Application messages use a 4-byte big-endian length prefix followed by CBOR payloads. Figure 7 summarizes measured wire sizes for representative messages. Heartbeat request and acknowledgment are 6 B each; a complete enrollment round trip totals 87 B of application payload; a minimal deploy round trip (33 B WASM module) totals 90 B. At the configured 25 s heartbeat period, steady-state control traffic is $12 \times (3600/25) = 1728$ B/h per device (≈ 1.69 KiB/h), excluding TLS record overhead.

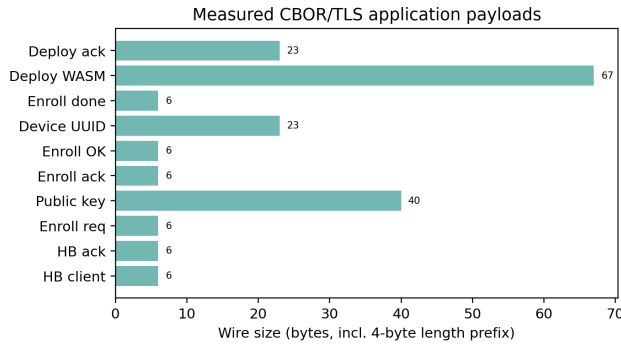


Figure 7: Application-layer wire sizes (4-byte length prefix + CBOR) for representative protocol messages.

7.3.2. Edge footprint

The device firmware (Zephyr, network stack, WAMR, TLS/CBOR client) occupies 394 KiB of flash and 264 KiB of static RAM on the STM32F746, comfortably within the board's 1 MiB flash and 320 KiB RAM budget. The flash figure combines 393,980 B of code (.text) and 9,380 B of initialized data (.data); the RAM figure adds 260,818 B of .bss. The unstripped ELF loaded by the emulator is larger (8.0 MB) only because it retains debug symbols and section metadata that are not present in the flashed image. The benchmark WASM module is 33 B. Embedded endpoints run no kubelet, container runtime, or Linux userspace.

Table 4

Orchestration model comparison.

Aspect	Device as K8s worker	Gateway-mediated WASM edge
Edge runtime	kubelet + container runtime + OS	Zephyr + WAMR (394 KiB flash, 264 KiB RAM)
Edge cluster agent RAM	100–350 MiB per node (literature)	none on device
Steady control traffic	CRI/container probes	1728 B/h per device
Fog mediation RAM	per-node agent	18.2 MiB shared gateway

7.3.3. Control-plane resources

Figure 8 reports mean Kubernetes pod usage ($n=5$ samples, one TLS-connected device). The gateway averaged 3.6 millicores CPU and 18.2 MiB RAM; the API server 91.0 millicores and 201.0 MiB; the application controller 44.8 millicores and 6.0 MiB. Registering up to 50 synthetic boards through the gateway HTTP registry did not increase gateway resident memory beyond 19.0 MiB, indicating lightweight per-device mediation state at the fog layer.

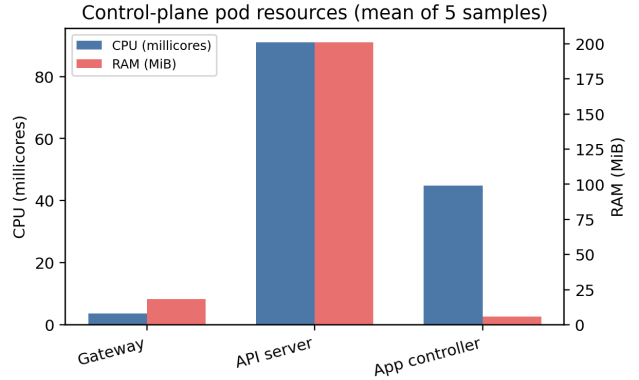


Figure 8: Mean CPU (millicores) and RAM (MiB) for control-plane pods during the sustainability measurement pass.

7.3.4. Architectural comparison

Table 4 contrasts gateway-mediated WASM orchestration with treating each constrained device as a Kubernetes worker. Edge Kubernetes deployments commonly require tens to hundreds of MiB of resident memory per worker for kubelet and container runtime before user workloads [39]. Our design concentrates orchestration in a shared gateway (≈ 18 MiB measured) while keeping cluster agents off the device.

8. Discussion

8.1. Architectural and RI Implications

The main architectural lesson is that the gateway should be treated as a semantic boundary, not as a transparent relay. Constrained devices can remain focused on protected communication, local execution, and evidence emission, while

the gateway handles identity validation, protocol translation, and status propagation toward Kubernetes. This separation avoids pushing cloud-facing semantics into firmware and gives the control plane a stable interface even if the south-bound protocol evolves.

The evaluation confirms that protocol success alone is not sufficient: enrollment, liveness, and deployment evidence must be interpreted consistently before they become trustworthy control-plane state. This supports the central design decision to separate desired state, gateway-derived evidence, and controller reconciliation. Future extensions such as multi-gateway placement, staged rollout, or retry policies should preserve this separation rather than allowing multiple components to write conflicting interpretations of device status.

For sustainable digital research infrastructures, the same separation creates explicit attachment points for energy and carbon policies. Placement decisions can be expressed as desired state, gateway and device observations can be treated as evidence, and sustainability controllers can consume the same status model without requiring constrained devices to run additional cloud-native agents. The measured baselines in Section 7.3—byte-scale southbound traffic, sub-20 MiB gateway mediation, and no kubelet on the edge—suggest that subsequent green-experimentation studies can focus on policy and instrumentation rather than on redesigning the orchestration substrate.

8.2. Validity and Scope

The validation targets system-level orchestration behavior rather than real-time performance, radio behavior, or sustainability optimization. Emulation and scripted trials are appropriate for exercising enrollment, deployment, status convergence, and reconciliation semantics, while physical boards, wireless links, and power meters require additional characterization. The results should therefore be interpreted as evidence that the orchestration model is coherent under the evaluated workflow.

External validity is bounded by the single-node K3s deployment, one gateway path, and a representative minimal workload. Larger deployments introduce gateway contention, distributed failure modes, network partitions, and device mobility. A supplementary scaling attempt with three Renode machines on the shared TAP test bed kept only one TLS session active in the gateway runtime view; fleet-scale energy characterization therefore requires isolated network segments or physical boards. Construct validity is also scoped: the study measures secure attachment, observable liveness, deployment completion, latency, wire sizes, firmware footprint, and control-plane RAM/CPU samples, while direct energy consumption and long-duration reliability remain outside the current evaluation.

Conclusion validity is strengthened by the transactional success criterion: enrollment, heartbeat visibility, and deployment must succeed within the same trial. This avoids overstating correctness from isolated protocol stages. Broader

claims about optimality, scalability, or comparative advantage require additional experiments against alternative orchestration designs and physical-device deployments.

8.3. Reproducibility

The evaluation is designed to be repeatable on a commodity Linux host with Docker and K3s. Renode makes the embedded side deterministic enough for regression-style validation, although it remains complementary to future physical-board experiments. Evidence traceability is maintained across multiple layers: gateway logs, API-visible resource states, and Kubernetes CRD status describe the same workflow from different perspectives, so that protocol success and control-plane convergence are not inferred from a single eventually consistent field.

9. Conclusion and Future Work

This paper presented a Kubernetes-native framework for managing embedded WebAssembly workloads across the Cloud–Fog–Edge continuum without requiring constrained devices to become Kubernetes nodes. The contribution integrates CRD-based intent, gateway-mediated TLS/CBOR communication, Zephyr firmware, WAMR execution, and evidence-driven reconciliation into a single verifiable operational path suitable for sustainable digital research infrastructures.

The experimental evaluation on a K3s test bed with emulated embedded targets confirms that the architecture supports a complete enrollment-to-deployment workflow under repeatable conditions, with 100/100 transactional success and sub-second mean end-to-end orchestration latency. Measured southbound control traffic, firmware footprint, and fog-layer resource use provide quantitative anchors for subsequent energy-aware placement and green-experimentation studies.

Future work will extend the platform toward multi-gateway deployments, fault injection, physical hardware, wattmeter-based energy measurements, carbon-intensity annotations, and larger device populations.

Data Availability Statement

The data supporting the findings of this study consist of deployment logs, Kubernetes resource states, controller and gateway traces, and protocol evidence from controlled experiments. Anonymized records, validation workflows, and accompanying documentation are available from the corresponding author upon reasonable request, subject to institutional policies and applicable confidentiality constraints.

Code Availability Statement

The source code and deployment assets associated with the framework are maintained in the public project repository at <https://github.com/lucadagati/k8s-wasm-edge-orchestration>.

Funding

This research received no external funding.

Conflict of Interest

The authors declare that they have no conflict of interest related to this work.

Author Contributions

Conceptualization: Giovanni Merlino, with input from Francesco Longo. Methodology: Luca D'Agati, Giuseppe Tricomi, Fabio Orazio Mirto, and Giovanni Merlino. Software: Luca D'Agati, with contributions from Giuseppe Tricomi and Fabio Orazio Mirto. Validation: Luca D'Agati, Giuseppe Tricomi, and Fabio Orazio Mirto. Investigation and data curation: Luca D'Agati, Giuseppe Tricomi, and Fabio Orazio Mirto. Writing—original draft: Luca D'Agati, with writing support from Giuseppe Tricomi and Fabio Orazio Mirto. Writing—review and editing: all authors. Supervision: Giovanni Merlino, with support from Francesco Longo.

References

- [1] Akri Authors, 2026. Akri: A kubernetes resource interface for the edge. <https://docs.akri.sh/>. Accessed: 2026-05-15.
- [2] Amazon Web Services, 2026. Freertos documentation. <https://www.freertos.org/>. Accessed: 2026-05-15.
- [3] Antmicro, 2026. Renode: Development framework for multi-node embedded networks. <https://renode.io/>. Accessed: 2026-05-15.
- [4] Baccelli, E., Hahm, O., Gunes, M., Wahlisch, M., Schmidt, T.C., 2013. RIOT OS: Towards an os for the internet of things, in: Proceedings of the IEEE Conference on Computer Communications Workshops, pp. 79–80.
- [5] Bonomi, F., Milito, R., Zhu, J., Addepalli, S., 2012. Fog computing and its role in the internet of things, in: Proceedings of the First Edition of the MCC Workshop on Mobile Cloud Computing, pp. 13–16.
- [6] Bormann, C., Hoffman, P., 2020. RFC 8949: Concise binary object representation (cbor). RFC 8949.
- [7] Bytecode Alliance, 2026a. Wasmtime: A fast and secure runtime for webassembly. <https://wasmtime.dev/>. Accessed: 2026-05-15.
- [8] Bytecode Alliance, 2026b. Webassembly system interface (wasi). <https://wasi.dev/>. Accessed: 2026-05-15.
- [9] Bytecode Alliance and WAMR Contributors, 2026. Webassembly micro runtime (wamr). <https://github.com/bytecodealliance/wasm-micro-runtime>. Accessed: 2026-05-15.
- [10] Canonical, 2026. Microk8s documentation. <https://microk8s.io/docs>. Accessed: 2026-05-15.
- [11] Chiang, M., Zhang, T., 2016. Fog and iot: An overview of research opportunities. IEEE Internet of Things Journal 3, 854–864.
- [12] containerd Contributors, 2026. runwasi: Webassembly runtime integration for containerd. <https://github.com/containerd/runwasi>. Accessed: 2026-05-15.
- [13] D'Agati, L., Tricomi, G., Arena, M., Longo, F., Puliafito, A., Merlino, G., 2025. Iot orchestration in the compute continuum: Integrating kubernetes with stack4things, in: 2025 IEEE 25th International Symposium on Cluster, Cloud and Internet Computing Workshops (CCGridW), pp. 140–147. doi:10.1109/CCGridW65158.2025.00028.
- [14] Dunkels, A., Gronvall, B., Voigt, T., 2004. Contiki – a lightweight and flexible operating system for tiny networked sensors, in: Proceedings of the 29th Annual IEEE International Conference on Local Computer Networks, pp. 455–462.
- [15] EdgeX Foundry, 2026. Edgex foundry documentation. <https://www.edgexfoundry.org/>. Accessed: 2026-05-15.
- [16] Fdida, S., Makris, N., Korakis, T., Bruno, R., Passarella, A., Andreou, P., Belter, B., Crettaz, C., Dabbous, W., Demchenko, Y., et al., 2022. Slices, a scientific instrument for the networking community. Computer Communications 193, 189–203.
- [17] Fermyon Technologies, 2026. Spin: Framework for webassembly applications. <https://www.fermyon.com/spin>. Accessed: 2026-05-15.
- [18] FIWARE Foundation, 2026. Fiware documentation. <https://www.fiware.org/>. Accessed: 2026-05-15.
- [19] Haas, A., Rossberg, A., et al., 2017. Bringing the web up to speed with webassembly, in: Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI), pp. 185–200.
- [20] Krustlet Authors, 2026. Krustlet: Kubernetes kubelet in rust for webassembly workloads. <https://krustlet.dev/>. Accessed: 2026-05-15.
- [21] KubeEdge Authors, 2026. Kubeedge documentation. <https://kubernetes.io/docs/concepts/extend-kubernetes/operator/>. Accessed: 2026-05-15.
- [22] Kubernetes Community, 2026. Operator pattern. <https://kubernetes.io/docs/concepts/extend-kubernetes/operator/>. Accessed: 2026-05-15.
- [23] Levis, P., Madden, S., Polastre, J., Szewczyk, R., Whitehouse, K., Woo, A., Gay, D., Hill, J., Welsh, M., Brewer, E., Culler, D., 2005. Tinyos: An operating system for sensor networks. Ambient Intelligence , 115–148.
- [24] OASIS, 2019. MQTT version 5.0. OASIS Standard.
- [25] oneM2M, 2026. onem2m technical specifications. <https://www.onem2m.org/technical/published-specifications>. Accessed: 2026-05-15.
- [26] OPC Foundation, 2026. Opc unified architecture specification. <https://opcfoundation.org/about/opc-technologies/opc-ua/>. Accessed: 2026-05-15.
- [27] Open Mobile Alliance, 2022. Lightweight machine to machine technical specification. OMA SpecWorks.
- [28] OpenYurt Authors, 2026. Openyurt: Extending kubernetes to edge computing. <https://openyurt.io/>. Accessed: 2026-05-15.
- [29] Osterlind, F., Dunkels, A., Eriksson, J., Finne, N., Voigt, T., 2006. Cross-level sensor network simulation with cooja, in: Proceedings of the 31st IEEE Conference on Local Computer Networks, pp. 641–648.
- [30] Rancher Labs, 2026. K3s: Lightweight kubernetes. <https://k3s.io/>. Accessed: 2026-05-15.
- [31] Rescorla, E., 2018. RFC 8446: The transport layer security (tls) protocol version 1.3. RFC 8446.
- [32] Rescorla, E., Tschofenig, H., Modadugu, N., 2022. RFC 9147: The datagram transport layer security (dtls) protocol version 1.3. RFC 9147.
- [33] Riley, G.F., Henderson, T.R., 2010. The ns-3 network simulator, in: Modeling and Tools for Network Simulation, Springer. pp. 15–34.
- [34] Satyanarayanan, M., 2017. The emergence of edge computing. Computer 50, 30–39.
- [35] Schaad, J., 2022. RFC 9052: Cbor object signing and encryption (cose): Structures and process. RFC 9052.
- [36] Selander, G., Mattsson, J., Palombini, F., Seitz, L., 2019. RFC 8613: Object security for constrained restful environments (oscore). RFC 8613.
- [37] Shelby, Z., Hartke, K., Bormann, C., 2014. RFC 7252: The constrained application protocol (coap). RFC 7252.
- [38] Shi, W., Cao, J., Zhang, Q., Li, Y., Xu, L., 2016. Edge computing: Vision and challenges. IEEE Internet of Things Journal 3, 637–646.
- [39] The Kubernetes Authors, 2026. Kubernetes documentation. <https://kubernetes.io/docs/>. Accessed: 2026-05-15.
- [40] The Zephyr Project, 2026. Zephyr project documentation. <https://docs.zephyrproject.org/>. Accessed: 2026-05-15.
- [41] Varga, A., Hornig, R., 2008. An overview of the omnet++ simulation environment. Proceedings of the 1st International Conference on Simulation Tools and Techniques for Communications, Networks and

Systems .

- [42] Wasm3 Authors, 2026. Wasm3: A fast webassembly interpreter. <https://github.com/wasm3/wasm3>. Accessed: 2026-05-15.
- [43] WasmEdge Authors, 2026. Wasmedge runtime. <https://wasmedge.org/>. Accessed: 2026-05-15.
- [44] World Wide Web Consortium, 2019. Webassembly core specification. W3C Recommendation.